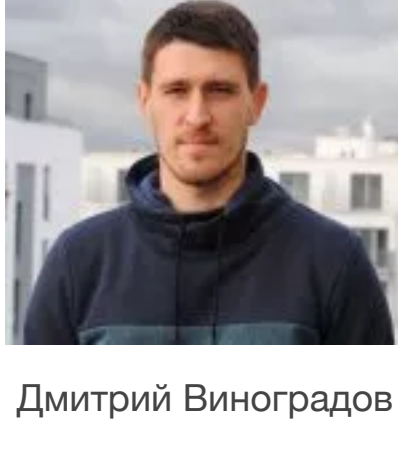


Language



Автор сайта



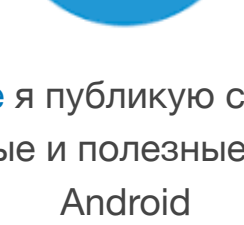
Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



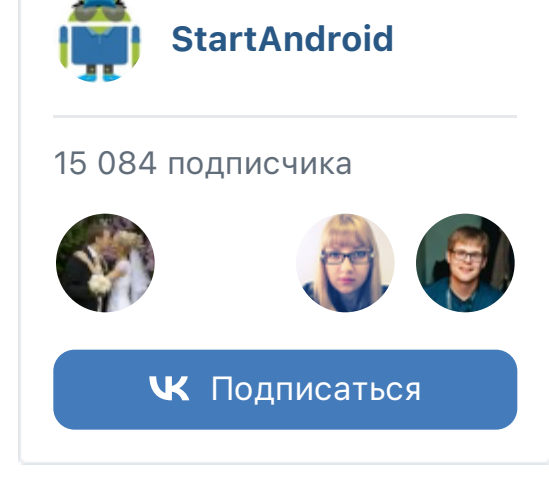
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 14. Fragment. ViewModel

13 августа 2021

В этом уроке передаем компонент от Activity фрагменту и настраиваем инджект ViewModel.

Фрагмент

Я просмотрел документацию по даггер на [developer.android.com](#). Кроме Hilt (его мы обсудим отдельно) я нашел там только одну тему, которую можно обсудить. Это передача компонента от Activity фрагменту.

Зачем вообще это нужно?

В примерах про скоре мы обсуждали, что Activity и его фрагментам может быть нужен один и тот же синглтон (например, репозиторий), время жизни которого ограничено этим Activity. Это реализуется с помощью сабкомпонента Activity.

Но, чтобы это работало, Activity и фрагмент должны иметь один и тот же экземпляр сабкомпонента. Только в этом случае они получат от него один и тот же экземпляр синглтона репозитория. Поэтому Activity должно как-то передать сабкомпонент фрагменту.

В противном случае, если фрагмент сам пойдет к даггеру за сабкомпонентом, даггер создаст ему новый экземпляр сабкомпонента. И фрагмент будет получать от него уже другой синглтон репозитория.

Давайте рассмотрим пример передачи сабкомпонента (или компонента) от Activity к фрагменту.

Кода фрагмента:

```
1 class LoginUsernameFragment: Fragment() {
2
3     // Fields that need to be injected by the login graph
4     @Inject lateinit var loginViewModel: LoginViewModel
5
6     override fun onAttach(context: Context) {
7         super.onAttach(context)
8
9
10        // Obtaining the login graph from LoginActivity and instantiate
11        // the @Inject fields with objects from the graph
12        (activity as LoginActivity).loginComponent.inject(this)
13
14        // Now you can access loginViewModel here and onCreateView too
15        // (shared instance with the Activity and the other Fragment)
16    }
```

В onAttach берем Activity и от него получаем компонент.

В результате и Activity и фрагмент будут иметь один и тот же экземпляр компонента (или сабкомпонента), а следовательно - одни и те же синглтоны.

Важное замечание! В LoginActivity.onCreate мы должны получать сабкомпонент до вызова super.onCreate. Иначе во фрагменте во время вызова onAttach компонент может быть еще недоступен.

ViewModel

Про ViewModel в документации особо ничего не говорят. Но дают [ссылку](#) на репозиторий с примером использования Dagger + ViewModel.

Давайте на простом примере разберемся, как там реализован инджект моделей.

Пусть у нас есть две ViewModel:

```
1 class ViewModel1(val networkUtils: NetworkUtils): ViewModel()
1 class ViewModel2(val dbHelper: DatabaseHelper): ViewModel()
```

Имена я выбрал самые простые, для примера вполне хватит. Пусть на вход они требуют NetworkUtils и DatabaseHelper.

Для создания этих моделей нам нужна фабрика:

```
1 class ViewModelFactory @Inject constructor(
2     val networkUtils: NetworkUtils,
3     val dbHelper: DatabaseHelper
4 ): ViewModelProvider.Factory {
5
6     override fun <T : ViewModel?> create(modelClass: Class<T>): T {
7         return when (modelClass) {
8             ViewModel1::class.java -> ViewModel1(networkUtils)
9             ViewModel2::class.java -> ViewModel2(databaseHelper)
10            else -> throw IllegalArgumentException("Unknown ViewModel class")
11        } as T
12    }
13
14 }
```

Конструктор фабрики помечаем Inject аннотацией. Даггер сам сможет создать ее для нас, объекты NetworkUtils и DatabaseHelper у него есть.

Добавляем get метод в сабкомпонент Activity, чтобы он возвращал нам фабрику:

```
1 @Subcomponent(modules = [MainModule::class])
2 interface MainComponent {
3
4     fun getViewModelFactory(): ViewModelFactory
5
6 }
```

В Activity вызываем этот get метод, получаем фабрику и используем ее для создания моделей:

```
1 mainComponent = (application as App).appComponent.getMainComponent().create(this)
2
3 val viewModelFactory = mainComponent.getViewModelFactory()
4
5 val model1 = ViewModelProvider(this, viewModelFactory).get(ViewModel1::class.java)
6 val model2 = ViewModelProvider(this, viewModelFactory).get(ViewModel2::class.java)
```

Это стандартный сценарий. Даггер создает для нас фабрику моделей и наполняет ее необходимыми объектами. А система потом использует эту фабрику, чтобы создавать модели.

В таком сценарии фабрика у нас все знает про каждую модель. И с каждой новой моделью нам надо вносить изменения в код фабрики. Плохая новость в том, что это не соответствует принципам SOLID.

Но есть и хорошая новость - мы можем это исправить. Создание всех моделей мы можем вынести в Provides методы в даггер-модуль. Кроме этого мы можем использовать аннотацию IntoMap, чтобы все эти модели собирались даггером в одну Map коллекцию. Эту коллекцию мы затем будем передавать в фабрику, чтобы она могла создавать модели.

Давайте реализуем это.

В модуле в Provides методах описываем создание моделей:

```
1 @Module
2 class ViewModelModule {
3
4     @IntoMap
5     @ViewModelKey(ViewModel1::class)
6     @Provides
7     fun provideViewModel1(networkUtils: NetworkUtils): ViewModel {
8         return ViewModel1(networkUtils)
9     }
10
11     @IntoMap
12     @ViewModelKey(ViewModel2::class)
13     @Provides
14     fun provideViewModel2(databaseHelper: DatabaseHelper): ViewModel {
15         return ViewModel2(databaseHelper)
16     }
17
18 }
```

Используем IntoMap, чтобы модели собирались в Map. В качестве ключа используем класс модели. Для этого надо будет создать свою ключ-аннотацию ViewModelKey:

```
1 @MapKey
2 @Retention(AnnotationRetention.RUNTIME)
3 annotation class ViewModelKey(val value: KClass<out ViewModel>)
```

Далее просим даггер передать получившийся Map в фабрику моделей:

```
1 class ViewModelFactory @Inject constructor(
2     private val viewModelProviders: @JvmSuppressWildcards Map<Class<out ViewModel>, Provider<ViewModel>>
3 ): ViewModelProvider.Factory {
4
5     override fun <T : ViewModel?> create(modelClass: Class<T>): T {
6         return viewModelProviders[modelClass]?.get() as T
7         ?: throw IllegalArgumentException("Unknown ViewModel class")
8     }
9
10 }
```

Теперь фабрика не надо будет ничего знать ни про модели, ни про объекты, которые им требуются. Все это ушло в модули даггера. Фабрика только достает из Map нужный провайдер, чтобы предоставить нам модель.

Обратите внимание, что в Map мы используем тип значения не ViewModel, а Provider<ViewModel>. Это нужно, чтобы фабрика создавала нам новую модель, а не использовала каждый раз один и тот же экземпляр.

Назад Вперед

Комментарии

- # **Dagger2 в многомодульном проекте** — Vladimir 21.08.2021 23:49
Не урока с примерами организации компонентов в многомодульном приложении, с передачей объектов через интерфейсы?
В инете примеры есть, но с таким подробным разбором и на русском не нашел..

Ответить
- # **RE: Dagger2 в многомодульном проекте** — Dmitry Vinogradov 22.08.2021 11:47
Многомодульность будет, еще не добрался до нее)

Ответить
- # **RE: Dagger2 в многомодульном проекте** — Vladimir 24.08.2021 01:21
Спасибо с буду ждать

Ответить
- # **Dagger2 в многомодульном проекте** — Vladimir 21.08.2021 23:49
Не хватает"

Ответить
- # **RE: Урок 14. Fragment. ViewModel** — Александр 14.09.2021 13:09
Что означает последняя фраза Урока про Provider?.. По умолчанию фабрика всегда создает ViewModel как синглтон?.. Нет ли какой-то ссылки на документацию по этому поводу? Заранее благодарю.

Ответить
- # **RE: Урок 14. Fragment. ViewModel** — Dmitry Vinogradov 16.09.2021 17:41
Если мы не используем Provider, то в Map будет лежать один конкретный экземпляр модели. И фабрика всегда и всем будет возвращать именно его. Т.е. каждый раз, когда мы открываем экран, мы получаем тот же экземпляр модели, что и в прошлый раз.
Provider же каждый раз будет создавать новую модель каждый раз, когда система будет просить ее у фабрики. Теперь при каждом открытии экрана мы будем получать новую модель.

Ответить
- # **Activity/Fragment component** — Александр 09.11.2021 11:26
(activity as LoginActivity).loginComponent.inject(this)
Не совсем понятно как это реализовано, потому что если в активности мы этот компонент инициализируем в onCreate(), то в onAttach() фрагмента этот компонент еще не будет инициализирован и будет null.

Ответить
- # **RE: Activity/Fragment component** — Dmitry Vinogradov 09.11.2021 12:01
Activity.onCreate должен вызываться до Fragment.onAttach
"Note that it's guaranteed that the Activity's onCreate is executed before the Fragment's."
<https://medium.com/androiddevelopers/the-android-lifecycle-cheat-sheet-part-iii-fragments-afc87d4f37fd>

Ответить
- # **RE: Activity/Fragment component** — Александр 09.11.2021 13:24
<https://startandroid.ru.ru/uroki/vse-uroki-spisokm/174-urok-104-android-3-fragments-lifecycle.html>
Я возможно где-то глубоко ошибаюсь. Но даже в Вашем уроке про жизненные циклы в примере с логами onAttach() фрагмента вызывается до onCreate() активности
Fragment1 onAttach
Fragment1 onCreate
Fragment1 onCreateView
Fragment2 onAttach
Fragment2 onCreate
Fragment2 onCreateView
MainActivity onCreate

Ответить
- # **RE: Activity/Fragment component** — Александр 09.11.2021 14:04
class LoginActivity: Activity() {
 // Reference to the Login graph
 lateinit var loginComponent: LoginComponent

 // Fields that need to be injected by the login graph
 @Inject lateinit var loginViewModel: LoginViewModel

 override fun onCreate(savedInstanceState: Bundle?) {
 // Creation of the login graph using the application graph
 loginComponent = (applicationContext as MyDaggerApplication)
 .appComponent.loginComponent().create()

 // Make Dagger instantiate @Inject fields in LoginActivity
 loginComponent.inject(this)

 // Now loginViewModel is available
 super.onCreate(savedInstanceState)
 }

Я думаю другим читателям данного курса будет полезно, если Вы вставите в раздел "Фрагмент" этот кусок кода с официальной документацией. Т.к. в Ваших примерах активности (например урок 10) каждый раз app компонент создается после вызова метода супер класса. А в официальных доках метод супер класса вызывается после. Из-за этого (например у меня) в onAttach() все время было null.

Ответить
- # **RE: Activity/Fragment component** — Dmitry Vinogradov 15.11.2021 22:44
Да, важное замечание, дополнил урок.
Спасибо!

Ответить
- # **RE: Урок 14. Fragment. ViewModel** — Николай 01.01.2022 21:51
После оптимизации фабрики, мы создаем viewModel через вызов метода create?
Правильно понимаю.

Ответить
- # **RE: Урок 14. Fragment. ViewModel** — Dmitry Vinogradov 24.01.2022 15:07
Получение модели не меняется
val model1 = ViewModelProvider(this, viewModelFactory).get(ViewModel1::class.java)

Просто мы код из фабрики раскидали по отдельным методам.

Ответить
- # **ViewModel** — Ильшат 09.02.2022 15:16
1) В AppComponent мы должны создавать наш SubComponent через:
fun getMainComponent(): MainComponent ???
2) Почему в коде Activity после getMainComponent() стоит .create(this)

Ответить
- # **RE: ViewModel** — Dmitry Vinogradov 14.02.2022 15:11
1) Т.к. сабкомпонент зависит от компонента и имеет доступ к его данным, то да, только так.
2) Это используется фабрика. Я не стал здесь приводить код фабрики в Main компоненте, чтобы лишний код не писать.

Посмотрите уроки 6 и 7, они как раз по этим темам.

Ответить
- # **Provide** — Alisher 22.04.2022 16:16
В методах @Provides модуль возвращает тип ViewModel, но в конструктор фабрики инжектиться Provide. Как даггер понимает что нужен не ViewModel а Provide?

Ответить
- # **remark** — Alisher 22.04.2022 16:18
То есть в фабрике инжектиться Provide

Ответить
- # **RE: remark** — Dmitry Vinogradov 22.04.2022 16:58
Именно по типу объекта, который должен быть заинджектен, Dagger понимает, что от него ждут не просто объект, а Provider объекта.
Далее он уже просто ищет в Provide методах такой объект, находит его и оборачивает код создания в Provider

Ответить
- # **ViewModelFactory** — Тимофей 23.12.2022 15:46
Через provide-метод создать фабрику, я так понимаю, нельзя?
@Provides
fun provideViewModelFactory(
 ...viewModelProviders: Map
): ViewModelFactory {
 ...return ViewModelFactory(viewModelProviders)
}

У меня жалуются на Map.

Ответить
- # **RE: ViewModelFactory** — Dmitry Vinogradov 26.12.2022 16:54
Должно быть можно, погуглите "kotlin dagger map". Там есть свои особенности. По-моему надо mutableMap использовать.
<https://stackoverflow.com/questions/62068696/dagger2-error-in-viewmodelfactory-when-change-mutablemap-to-map>

Ответить
- # **@JvmSuppressWildcards** — Даниил 14.11.2023 10:41
Вы скорее всего забыли указать @JvmSuppressWildcards

Ответить

🔄 Обновить список комментариев
📖 RSS лента комментариев этой записи

Добавить комментарий

Тема

👍👎😄😁😂😃😅😆😇😈😉😊😋😌😍😎😏😐😑😒😓😔😕😖😗😘😙😚😛😜😝😞😟😠😡😢😣😤😥😦😧😨😩😪😫😬😭😮😯😰😱😲😳😴😵😶😷😸😹😺😻😼😽😾😿😺

Отправить

3 Comments

